

Recetas del Ticket — Documentación técnica

Versión del documento: 1.0 · **Fecha:** 6 de agosto de 2026

Repositorio: [hparedes95/recetas_ticket](https://github.com/hparedes95/recetas_ticket) · rama `claude/recipe-app-shopping-ticket-ng8q5o`

Aplicación publicada: https://hparedes95.github.io/recetas_ticket/

1. Qué es

Aplicación móvil que convierte la compra semanal en planes de comidas. Funciona en dos sentidos:

- **Flujo ticket -> menú:** se introduce el ticket del supermercado (foto, texto pegado o modo compra) y la app propone menús con lo que hay en casa.
- **Flujo menú -> compra:** la app propone un menú semanal completo y genera la lista de la compra con cantidades.

Uso familiar: varios miembros con sus propios gustos, un único menú conjunto, y sincronización opcional entre móviles.

Cifras del proyecto

Concepto	Valor
Ficheros fuente (TS/TSX)	61
Líneas de código	11.682
Recetas en catálogo	106
Ingredientes reconocidos	150
Tests automatizados	95 (14 suites)
Commits	40

2. Entorno y dependencias

Componente	Versión	Motivo
Expo SDK	54	Es la versión que soporta la app Expo Go publicada en la App Store. Subir el SDK rompe la apertura en el móvil.
React Native	0.81.5	Ligada al SDK 54.
React	19.1.0	Ligada al SDK 54.
TypeScript	5.9.2	<code>strict: true.</code>
Jest	jest-expo 54	Tests del motor en entorno node.

Restricción de plataforma: el motor JavaScript de React Native (Hermes) no admite paquetes que importen módulos de Node. Por eso las integraciones externas (API de Claude, Firebase) se hacen con fetch contra sus APIs REST en lugar de con sus SDK oficiales.

3. Arquitectura

App.tsx	Punto de entrada (ErrorBoundary + providers + navegación)
src/	
types/index.ts	Tipos centrales (Product, Recipe, MealPlan, Profile...)
theme/index.ts	Colores, espaciado y metadatos de objetivos
data/	
ingredients.ts	Diccionario de 150 ingredientes + matcher del ticket
aliases.ts	Sinónimos regionales, catalán, inglés y comerciales
recipes.ts	Catálogo de 106 recetas con macros por ración
culinary.ts	Metadatos culinarios (rol, técnicas, afinidades)
nutrition.ts	Nutrición por 100 g y conversión de unidades
dietary.ts	Estándares AESAN / dieta mediterránea
catalog.ts	Agrupación por categorías para el modo compra
engine/	
normalize.ts	Canonicalización del texto del ticket
ticketParser.ts	Texto del ticket -> productos
generator.ts	Generación de recetas por plantillas
weekly.ts	Planificador semanal (sin repetir, con cuotas)
planner.ts	Planes desde la despensa
recommend.ts	Flujo "recomiéndame la semana" + lista de la compra
suggest.ts	Orquestador de sugerencias (scoring, MMR)
history.ts / mmr.ts	Anti-repetición y diversidad
household.ts	Perfiles familiares -> preferencias efectivas
sync.ts	Sincronización entre móviles (Firebase REST)
config.ts	Pesos y parámetros ajustables del motor
aiRecipes.ts / aiTicket.ts	Integración opcional con la API de Claude
__tests__/	95 tests
context/AppContext.tsx	Estado global + persistencia (AsyncStorage)
navigation/	Tabs (Inicio, Despensa, Planes, Compra, Ajustes) + stack
screens/	Pantallas de la app
components/	Componentes reutilizables y ErrorBoundary

Principio de diseño: todo el motor son funciones puras sin dependencias de React Native. Esto permite testearlo en Node y fue lo que hizo barata la funcionalidad familiar: para generar el menú de una familia basta con llamar a las mismas funciones con unas preferencias distintas.

4. Motor de recetas

4.1 Canonicalización del ticket

El texto libre del ticket se convierte en claves de ingrediente:

1. Se descartan líneas que no son productos (totales, IVA, dirección, droguería).
2. Se limpia el ruido (precios, gramajes, códigos de artículo).
3. Se busca coincidencia por palabras clave, ganando la más específica.
4. Si no hay coincidencia, se aplican los alias (regionales, catalán, inglés, cortes comerciales) con normalización de singular/plural.
5. Lo no reconocido se conserva como otro: para no perder productos.

4.2 Generación por plantillas

El motor no elige de una lista cerrada: instancia plantillas de estructura (base + proteína + verdura + aromático + ácido/grasa + técnica) con los ingredientes reales disponibles, filtrando por técnicas compatibles y afinidades culinarias. Escribe los pasos según la técnica, con tiempos y temperaturas reales, y recalcula los macros sumando desde la tabla nutricional.

Validador obligatorio: ninguna receta se muestra si contiene un ingrediente que no existe en el diccionario.

4.3 Anti-repetición

- Historial de sugerencias con firma estable por receta.
- Penalización por recencia con decaimiento exponencial sobre receta, proteína y técnica.
- Cooldown duro configurable.
- Selección del lote por MMR (relevancia menos similitud), no por top-N.

4.4 Planificación semanal

Reglas que cumple el menú generado:

Regla	Implementación
Ninguna comida se repite	Registro global de la semana: 21 comidas, 21 recetas distintas
Platos sencillos	Máximo 40 minutos y 5 pasos
Cena ligera	Se descartan platos con base de pasta/arroz/patata y los de más de 600 kcal
Legumbres ≥ 4 /semana	Cuota semanal (objetivo 5)
Pescado 2-3/semana	Cuota semanal
Carne priorizando aves	Tope de carne roja (máx. 2)
Objetivo de calorías	Selección de recetas + factor de ración natural (0,85–1,2)

Fuente de los criterios: AESAN 2022 (recomendaciones dietéticas sostenibles) y pirámide de la Fundación Dieta Mediterránea.

4.5 Ajuste del comportamiento

Los pesos del scoring están centralizados en `src/engine/config.ts`:

```
score = w.coverage·cobertura + w.affinity·afinidad + w.novelty·novedad
        - w.repetition·penalización_recencia - w.missing·nº_faltantes
```

5. Uso familiar

Cada miembro tiene su ficha con gustos (favoritos y descartados), restricciones, edad y calorías. El menú es **uno solo** para toda la familia:

- Los alimentos descartados por **cualquier** miembro se excluyen del menú.
- Las restricciones dietéticas se suman (si un miembro es celíaco, el menú es sin gluten).

- Los favoritos de todos se intercalan para que no predominen los de una sola persona.
- Cada miembro recibe su propia ración del mismo plato (un niño equivale a 0,55 raciones de adulto) y la lista de la compra escala con el tamaño real de la familia.
- Un contador de "platos posibles" avisa cuando la acumulación de vetos deja el menú sin variedad.

Sincronización entre móviles

Opcional, mediante Firebase Realtime Database del propio usuario y un código de familia de 16 caracteres. Se sincroniza al abrir la app y cada 30 segundos.

Estrategia de fusión: cada sección del estado se resuelve por marca de tiempo (gana la más reciente), salvo la lista de la compra, que se fusiona ítem a ítem para que dos personas puedan tachar productos a la vez sin perder cambios.

Nota de seguridad: cualquiera que tenga el código de familia puede leer y escribir los datos. No hay autenticación por usuario.

6. Runbook: entorno de desarrollo

Objetivo: levantar la app en local y verla en el móvil.

Requisitos previos: Node.js instalado, la app Expo Go en el móvil, y móvil y ordenador en la misma red Wi-Fi.

Procedimiento

1. Clona el repositorio y entra en la carpeta:

```
git clone https://github.com/hparedes95/recetas_ticket.git
cd recetas_ticket
```

2. Instala las dependencias:

```
npm install
```

3. Arranca el servidor de desarrollo:

```
npx expo start
```

4. Escanea con la cámara del móvil el código QR que aparece.

Verificación

- La app se abre en Expo Go y cada cambio de código se recarga en segundos.

Si algo falla

- **"La versión es incompatible" en Expo Go** -> el SDK del proyecto no coincide con el de la app Expo Go instalada. No subas el SDK sin comprobar antes cuál soporta Expo Go.
 - **Móvil y ordenador en redes distintas** -> arranca con `npx expo start --tunnel`.
 - **Errores raros tras cambiar dependencias** -> `npx expo start -c` para limpiar la caché.
-

7. Runbook: verificación y despliegue

Objetivo: publicar cambios en la web sin romper producción.

Requisitos previos: permiso de escritura en el repositorio.

Procedimiento

1. Comprueba los tipos:

```
npx tsc --noEmit
```

2. Ejecuta los tests:

```
npm test
```

3. Compila la web en local si quieres revisarla antes:

```
npm run build
```

4. Sube los cambios a la rama de trabajo:

```
git push -u origin claude/recipe-app-shopping-ticket-ng8q5o
```

Verificación

- El workflow `deploy-pages.yml` ejecuta comprobación de tipos, tests y compilación antes de publicar. Si algo falla, **no** se despliega.
- El despliegue tarda aproximadamente un minuto.
- Comprueba el resultado en https://hparedes95.github.io/recetas_ticket/ recargando la página.

Si algo falla

- **El workflow sale en rojo** -> revisa el paso que falló en la pestaña Actions del repositorio; los fallos habituales son de tipos o de tests.
- **La web no muestra los cambios** -> la caché del `index.html` tarda unos minutos; fuerza recarga o cierra y reabre la app.

8. Runbook: instalar la app en el móvil

Objetivo: usar la app en el teléfono sin ordenador ni Expo Go.

Procedimiento

1. Abre https://hparedes95.github.io/recetas_ticket/ en Safari (iPhone) o Chrome (Android).
2. Pulsa el botón **Compartir**.
3. Selecciona **Añadir a pantalla de inicio**.

Verificación

- Aparece el icono "Recetas" y la app se abre a pantalla completa.
- Las actualizaciones se aplican solas al abrir la app: no hay que reinstalar el icono.

Si algo falla

- **No aparecen los cambios recientes** -> cierra la app del todo y vuelve a abrirla.

Limitaciones conocidas

- Al ser una aplicación web, la cámara nativa no está disponible: la foto del ticket usa el selector de archivos del navegador.
- Requiere conexión para abrirse (no hay service worker de uso sin conexión).

- Los datos se guardan en el navegador del dispositivo; sin sincronización activada, no se comparten entre móviles.
-

9. Runbook: activar la sincronización familiar

Objetivo: que varios móviles compartan planes, despensa y lista de la compra.

Requisitos previos: una cuenta de Google.

Procedimiento

1. Entra en <https://console.firebase.google.com> y crea un proyecto.
2. En el menú lateral, abre **Realtime Database** y créala.
3. Copia la URL de la base de datos.
4. En **Realtime Database -> Reglas**, publica estas reglas (el modo de prueba caduca a los 30 días y deja la base abierta):

```
{
  "rules": {
    "familias": {
      "$codigo": {
        ".read": "$codigo.length >= 16",
        ".write": "$codigo.length >= 16"
      }
    }
  }
}
```

5. En la app: **Ajustes -> Mi familia -> Compartir con mi familia**, pega la URL y pulsa

Crear familia.

6. Comparte el código generado con el resto de la familia mediante **Invitar a alguien**.
7. En el otro móvil: misma pantalla, pega la URL y el código, y pulsa **Unirme a la familia**.

Verificación

- La pantalla muestra "Sincronizado" con la hora de la última sincronización.
- Un cambio hecho en un móvil aparece en el otro en menos de 30 segundos.

Si algo falla

- **"La base de datos no permite el acceso"** -> las reglas no están publicadas o el modo de prueba ha caducado.
 - **"Error de sincronización (404)"** -> la URL de la base de datos tiene una errata.
 - **"Tardó demasiado"** -> problema de conexión del dispositivo.
-

10. Registro de cambios

Evolución del proyecto en 40 commits, agrupada por bloques.

Base de la aplicación

- Creación de la app con Expo, React Native y TypeScript: tipos, tema, catálogo de datos, motor inicial, estado con persistencia, navegación y pantallas.
- Ajuste a **Expo SDK 54** por compatibilidad con la app Expo Go de la App Store.

Objetivos nutricionales

- Selección de calorías diarias y reparto de macros.
- **Calorías exactas sin inflar raciones:** el motor elige la combinación de recetas cuyo total se acerca al objetivo, en lugar de agrandar las porciones.
- Integración opcional con la API de Claude para generar recetas a medida.

Detección del ticket

- Ampliación del diccionario de ingredientes y filtrado del ruido del ticket.
- Retirada de la importación por PDF.
- Capa de canonicalización con alias regionales, catalán, inglés y variantes comerciales.
- Matcher optimizado (palabras clave precalculadas).

Publicación

- Exportación web y empaquetado como PWA (metadatos de instalación e iconos).
- Publicación en **GitHub Pages** con despliegue automático en cada push.
- Corrección de los diálogos en web: `Alert` de React Native no funciona en navegador, por lo que botones como "Vaciar" no hacían nada.

Rediseño del motor

- Metadatos culinarios de los ingredientes y tabla nutricional por 100 g.
- Motor generativo por plantillas con validador de factibilidad.
- Anti-repetición (historial, recencia, cooldown) y diversidad de lote por MMR.
- Configuración de pesos centralizada.
- Tests de aceptación con 60 tickets simulados y benchmark de rendimiento.

Calidad

- Infraestructura de tests con jest-expo.
- Revisiones de código que detectaron y corrigieron: la leche de coco desaparecida de la lista de la compra, nombres de ingrediente inconsistentes, y el cuscús etiquetado erróneamente como "sin gluten".
- Error boundary, memoización del contexto y ejecución de tests en CI antes de desplegar.

Flujo "recomiéndame la semana"

- Menú semanal completo sin depender de la despensa, con lista de la compra con cantidades.
- Menú sin repetir ninguna comida, con platos sencillos y cuotas dietéticas.
- Cenas ligeras y selector de alimentos favoritos y descartados.

Uso familiar

- Perfiles de familia con gustos propios y preferencias efectivas del hogar.

- Migración del estado de usuario único a multi-perfil sin pérdida de datos.
- Pantallas de familia y gustos por persona.
- Sincronización entre móviles con Firebase y código de familia.

11. Convenciones del proyecto

- Todo el texto de cara al usuario va en español.
- Las claves de ingrediente de las recetas deben existir en el diccionario `INGREDIENTS`. Hay un test que falla si no se cumple.
- Los básicos de despensa (aceite, sal, ajo...) llevan `staple: true` y no penalizan la cobertura ni entran en la lista de la compra.
- La lógica del motor no depende de React Native, para poder testearla en Node.
- Las claves de API (Anthropic) se guardan solo en el dispositivo y nunca se suben al repositorio ni se registran en logs.

12. Riesgos y limitaciones conocidas

Riesgo	Estado
Acumulación de vetos en familias grandes deja el menú sin variedad	Mitigado con contador de platos posibles y aviso
Quien tenga el código de familia accede a los datos	Documentado; el código es largo y aleatorio
No hay uso sin conexión (falta service worker)	Pendiente
La foto del ticket no se lee automáticamente (falta OCR)	Pendiente
Distribución solo como web/PWA, no en App Store	Requiere cuenta de Apple Developer